PROJECT REPORT

Flight Delay Prediction

NYC Flights Dataset (2013)

Binary Classification Using Scheduled Flight Information

Team	T029
Section	E
Prediction cutoff	6:00 PM on the previous day
Dataset	NYC Flights, 2013

Team

Members

Aarsh Bhatnagar	2401010007
Anurag Kumar Tiwari	2401020009
Nav Prabhat Singh	2401010293
Ashish Kumar Yadav	2401020015
Shivansh Tiwari	2401020108

Abstract

This project develops a binary classifier for predicting whether a flight departing from JFK, LaGuardia (LGA), or Newark Liberty (EWR) in 2013 will arrive at least 15 minutes late. The prediction cutoff is 6:00 PM on the previous day, so predictors must reflect information available by that time. Of 336,776 source records, 9,430 have no observed arrival-delay label and are excluded, leaving 327,346 labelled flights; 24.47% are delayed. The workflow combines exploratory data analysis, feature engineering, leakage controls, preprocessing pipelines, and chronological training, validation, and testing. A prior-probability Dummy Classifier, Logistic Regression, and XGBoost are compared using Average Precision (AP), with ROC-AUC as a secondary ranking metric. Logistic Regression is selected by validation AP (0.2348 versus 0.2301 for XGBoost). At a validation-selected threshold of 0.2611, the model achieves test AP 0.3359, ROC-AUC 0.6070, precision 0.3415, recall 0.4456, and F1-score 0.3867. These results indicate limited predictive signal rather than deployment-ready forecasting. Important limitations include temporal variation, absent weather observations, exclusion of flights without observed arrival delay, and the need to verify feature availability at the stated cutoff.

Headline result

Selected model: Logistic Regression | Test AP: **0.3359** | ROC-AUC: **0.6070** | F1: **0.3867**.
The model has measurable but limited ranking ability; it should not be described as deployment-ready.

Keywords: flight delay prediction, binary classification, Logistic Regression, XGBoost, temporal validation, data leakage, class imbalance.

Contents

Abstract **i**

1 Introduction **1**

1.1 Objectives . . . 1

2 Problem Definition and Key Terms **2**

2.1 Definitions and notation: quick reference . . . 2

2.2 Research question . . . 2

3 Related Work and Theoretical Rationale **3**

3.1 Logistic Regression . . . 3

3.2 XGBoost . . . 3

3.3 Why chronological evaluation matters . . . 3

4 Dataset and Exploratory Data Analysis **4**

4.1 Dataset profile and target cleaning . . . 4

4.2 Patterns identified in the EDA . . . 4

4.3 Leakage audit . . . 4

5 Feature Engineering and Preprocessing **7**

5.1 Predictor groups . . . 7

5.2 Preprocessing pipeline . . . 7

6 Experimental Protocol **8**

6.1 Chronological partitioning . . . 8

6.2 Models and selection rule . . . 8

6.3 Metric definitions . . . 8

7 Results **9**

7.1 Validation model comparison . . . 9

7.2 Held-out test performance . . . 9

7.3 Feature reliance . . . 10

8 Discussion **11**

8.1 Interpreting the comparison . . . 11

8.2 Temporal variation and operational meaning . . . 11

9 Reproducibility and Implementation **12**

10 Limitations and Future Work **13**

11 Conclusion **14**

A Reproducibility Checklist **15**

References **16**

CHAPTER 1

Introduction

Flight delays affect passengers, airline schedules, airport resources, and onward connections. A risk estimate made before departure could support planning, but it is useful only if it relies on information genuinely available at the time the prediction is made. Historical operational datasets contain both scheduled attributes and outcomes observed after a flight operates; mixing these can make a model appear more accurate than it would be in real use.

This project studies flights departing from New York City's JFK, LGA, and EWR airports during 2013. The target is whether a flight arrives at least 15 minutes late. The report focuses on a clear prediction-time assumption, chronological evaluation, and comparison of a simple baseline with two machine-learning models. It does not claim that a deep-learning model has been trained in the evaluated experiment.

1.1 Objectives

The main objectives are to: (1) inspect the dataset and delay-label distribution; (2) identify temporal and carrier-level patterns; (3) construct schedule and calendar features; (4) prevent target and post-departure leakage; (5) compare a baseline, Logistic Regression, and XGBoost; (6) choose the model and threshold using validation data; and (7) evaluate the selected model on a later, untouched test period.

CHAPTER 2

Problem Definition and Key Terms

Let D_i be the observed arrival delay in minutes for flight i. For records with a known arrival delay, the binary target is

$$y_i = \begin{cases} 1, & D_i \geq 15 \text{ minutes}, \\ 0, & D_i < 15 \text{ minutes}. \end{cases} \tag{2.1}$$

The task is to estimate $P(y_i = 1 \mid x_i)$ from a feature vector x_i that is available by 6:00 PM on the previous day. Actual departure time, departure delay, actual arrival time, air time, and arrival delay are therefore prohibited as predictors. Arrival delay is used only to construct the label.

Prediction-time rule

Only information available by **6:00 PM on the previous day** may be used. Actual departure/arrival times, departure delay, air time, and arrival delay are outcomes or post-departure information and must not be predictors.

2.1 Definitions and notation: quick reference

Term	Meaning in this project
Prediction cutoff	Assumed time the prediction is made: 6:00 PM on the day before departure.
Data leakage	Use of information unavailable at prediction time, or information from validation/test data during model development.
Chronological split	Training observations precede validation observations, which precede the held-out test period.
Training set	Earliest time partition used to fit model parameters and preprocessing.
Validation set	Later partition used for model comparison and threshold selection; not the final performance estimate.
Test set / holdout	Latest partition reserved for final evaluation.
Class prevalence	Fraction of examples labelled delayed; for a constant-score classifier, it is the baseline AP on that split.
Average Precision (AP)	Summary of precision over recall increments; the main ranking metric in this project.
ROC-AUC	Threshold-independent measure of ranking discrimination; 0.5 corresponds to chance-level ranking.
Precision and recall	Precision is the fraction of flagged flights that are delayed; recall is the fraction of delayed flights detected.
F1-score	Harmonic mean of precision and recall; depends on the classification threshold.
Permutation importance	Change in a validation metric after shuffling a feature; indicates predictive reliance, not causality.

2.2 Research question

Do scheduled-flight and calendar attributes improve ranking over a constant-score baseline on a later period? Findings should be interpreted as evidence for this dataset and year, not a universal claim about all flights.

CHAPTER 3

Related Work and Theoretical Rationale

Flight-delay prediction has been studied as classification, regression, and spatiotemporal forecasting. Reviews identify growing use of machine learning while highlighting issues such as external data integration, uncertainty, and deployment realism [3]. Benchmarking work also cautions that a more complex algorithm does not automatically outperform a simpler one; feature validity and evaluation design remain central [4]. These findings motivate a controlled comparison rather than assuming that boosting must win.

3.1 Logistic Regression

Logistic Regression maps a linear score to an estimated probability:

$$P(y_i = 1 \mid x_i) = \sigma(\beta_0 + \boldsymbol{\beta}^T x_i), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}. \tag{3.1}$$

Its regularized form is a practical benchmark for tabular data and is relatively easy to inspect. However, its log-odds are linear in the encoded inputs unless nonlinear transformations or interactions are supplied explicitly [8].

3.2 XGBoost

XGBoost builds an additive ensemble of decision trees and can represent nonlinear effects and feature interactions [5]. The candidate configuration uses 400 estimators, maximum depth 5, learning rate 0.05, and row and column subsampling of 0.8. Greater flexibility may help capture complex relationships, but it can also increase overfitting or sensitivity to hyperparameters. Validation performance, not model complexity alone, determines whether it is preferred.

3.3 Why chronological evaluation matters

Random splits can distribute similar dates and repeated service patterns across training and testing, making a future-prediction task appear easier. A chronological split more closely approximates deployment into a later period. It does not eliminate distribution shift, and a single split remains sensitive to the specific months used; repeated rolling-origin evaluation would provide stronger evidence of stability.

CHAPTER 4

Dataset and Exploratory Data Analysis

4.1 Dataset profile and target cleaning

The source dataset contains 336,776 records and 21 columns for flights departing JFK, LGA, and EWR in 2013 [1], [2], [9]. The EDA notebook reports 9,430 missing values for `arr_delay`. Removing these rows leaves 327,346 labelled flights, approximately 97.2% of the source records. The observed delayed-flight prevalence is 24.47%. Cancellations and diversions are not separately modelled, and excluding records without arrival-delay labels may introduce selection bias because these records are not necessarily random.

Because delayed flights are the minority class, accuracy alone can be misleading. A rule that predicts every flight as not delayed would achieve relatively high accuracy while detecting no delays. AP and recall are therefore reported alongside ROC-AUC and threshold-dependent metrics.

4.2 Patterns identified in the EDA

The recorded analysis shows that delay rates generally increase later in the day: early-morning rates are below 15% in the displayed hourly analysis, while late-evening rates exceed 30%. Saturday has the lowest weekday rate (about 17%) and Thursday the highest (about 28%). Carrier rates also vary, and the carrier chart is restricted to carriers with at least 1,000 labelled flights to reduce instability in small groups. These associations are descriptive, not causal; season, schedule composition, and operational conditions may contribute.

What the EDA suggests

Delay rates rise later in the day in this dataset; Saturday is the lowest-rate weekday (about 17%) and Thursday the highest (about 28%). These patterns support schedule/calendar features, but do not prove causation.

4.3 Leakage audit

The EDA notebook reports that `dep_delay` alone produces ROC-AUC of approximately 0.895 for the arrival-delay target, and that about 79.9% of flights departing at least 15 minutes late subsequently meet the arrival-delay label. These figures illustrate the strong relationship between departure and arrival delay, but actual departure delay is known only after departure. It must therefore be excluded from a model intended to predict the previous evening. The same restriction applies to actual departure time, arrival time, and air time.

label, 527,540 flights remain. The positive class is defined
east 15 minutes; its prevalence is 24.47%. This class distribu
verage Precision alongside ROC-AUC rather than relying on a

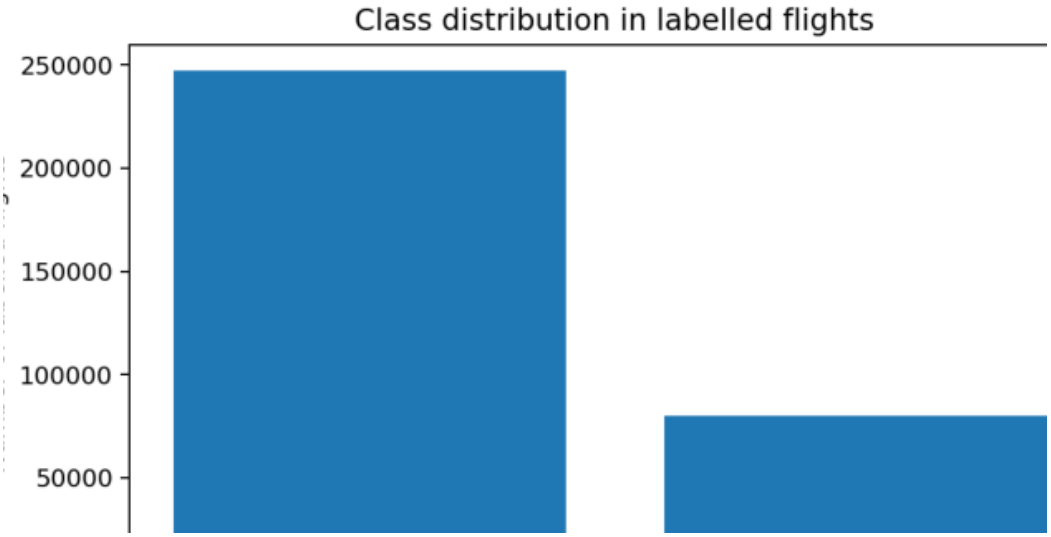


Figure 4.1: Class distribution among flights with an observed arrival-delay label.

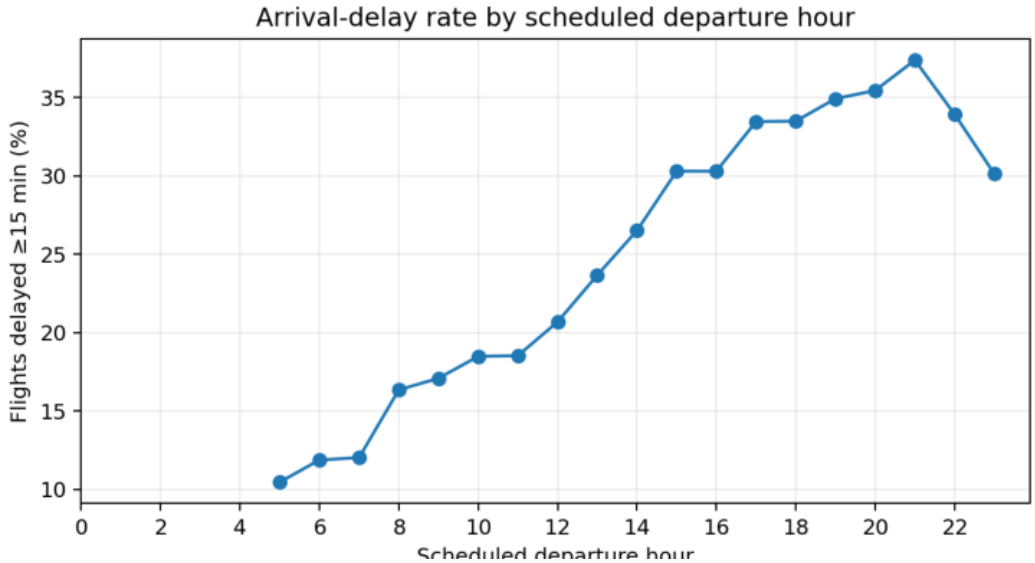


Figure 4.2: Observed arrival-delay rate by scheduled departure hour. This is descriptive analysis, not a causal effect.

sis, not a causal effect.

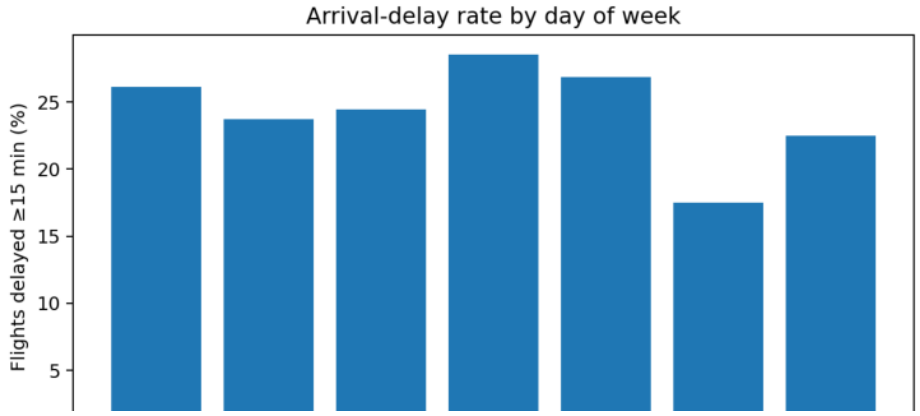


Figure 4.3: Observed arrival-delay rate by weekday. Differences may reflect seasonality, schedule composition, or operational conditions.

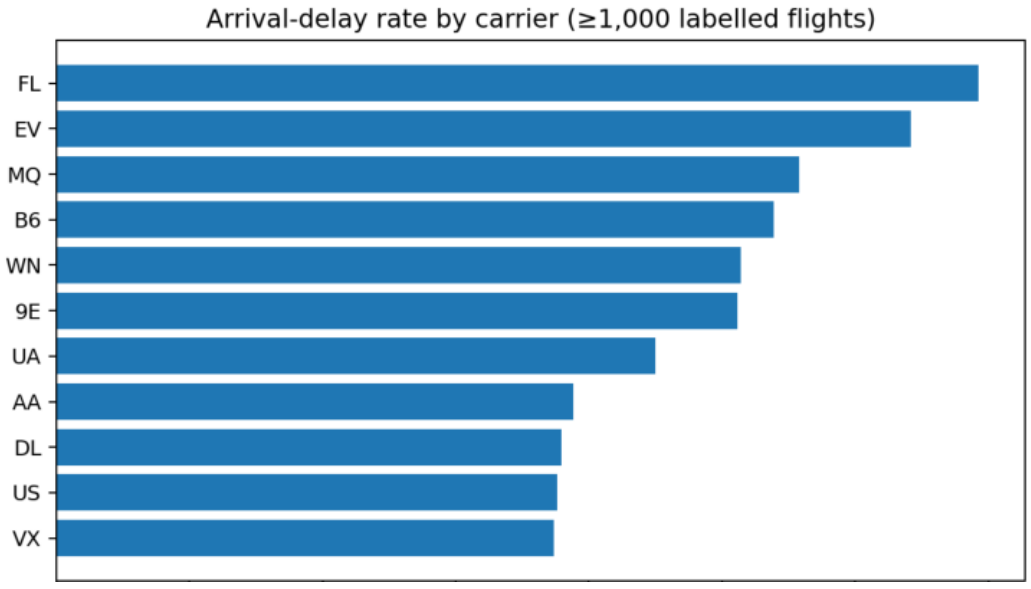


Figure 4.4: Observed arrival-delay rate by carrier, restricted to carriers with at least 1,000 labelled flights.

CHAPTER 5

Feature Engineering and Preprocessing

5.1 Predictor groups

The modelling notebook uses categorical features for carrier, origin airport, grouped destination, and flight number. Numeric and schedule-derived inputs include distance, scheduled departure and arrival minutes, and the count of scheduled flights at the same origin and hour. Cyclical representations are used for scheduled hour, weekday, and day of year. These features aim to represent recurring schedule patterns without importing realized operating outcomes.

Periodic variables are encoded with sine and cosine:

$$x_{\sin} = \sin\left(\frac{2\pi x}{P}\right), \qquad x_{\cos} = \cos\left(\frac{2\pi x}{P}\right), \tag{5.1}$$

where P is the period (24 for hour, 7 for weekday, and 365 for day of year in 2013). This keeps the end and start of a cycle close in feature space. Rare destination values are grouped into an OTHER category based on training data.

5.2 Preprocessing pipeline

Numeric values are imputed with the training median and standardized using training-set statistics. Categorical values are imputed with the training most frequent value and one-hot encoded; unknown categories are ignored and infrequent categories are grouped at a minimum frequency of 20. Each transformation is fitted on training data only, then applied unchanged to validation and test data [8]. This reduces the risk that held-out data influence imputation, scaling, or encoding.

The feature `scheduled_origin_hour_count` is a schedule-based proxy, not a direct observation of runway congestion, weather, or real-time airport conditions. Its use is defensible only if the relevant schedule was available by the stated cutoff and the count was constructed without outcome or future information. Flight number may capture recurring service patterns but may also encourage memorization, so an ablation test without this feature is recommended.

CHAPTER 6

Experimental Protocol

6.1 Chronological partitioning

Data are split by calendar month rather than randomly shuffled. January–August is used for training, September–October for validation, and November–December for testing. The validation period supports model and threshold selection; the test period remains untouched until final evaluation.

Table 6.1: Chronological data partitions

Subset	Rows	Delayed	Rate	Purpose
Train (Jan–Aug)	217,727	57,423	26.37%	Fit preprocessing and model parameters
Validation (Sep–Oct)	55,628	8,731	15.70%	Select model and threshold
Test (Nov–Dec)	53,991	13,946	25.83%	Final held-out evaluation

Important data-quality observation

Validation delay prevalence is only **15.70%**, compared with **26.37%** in training and **25.83%** in testing. This temporal variation can affect model selection and threshold transfer.

Validation prevalence is about 10 percentage points lower than training and test prevalence. Since AP depends on class prevalence, results should be interpreted with the base rate of each split. Chronological splitting avoids temporal mixing but does not eliminate distribution shift.

6.2 Models and selection rule

The candidates are a prior-probability Dummy Classifier, Logistic Regression, and XGBoost. Average Precision is the primary selection metric because delayed flights are the less prevalent class; ROC-AUC is a complementary ranking measure. The selected threshold is chosen by maximizing validation F1 and is frozen before test evaluation. Tuning a threshold against test labels would leak information from the holdout and make the reported test performance optimistic.

6.3 Metric definitions

For true positives (TP), false positives (FP), and false negatives (FN),

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F1 = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{6.1}$$

AP summarizes precision across recall increments. Its constant-score baseline is the positive-class prevalence for that same split. AP lift is defined as AP divided by prevalence; values above 1 indicate performance above the constant-score baseline under this definition. Accuracy is included as a secondary diagnostic because it can conceal missed positive cases.

CHAPTER 7

Results

7.1 Validation model comparison

Table 7.1: Validation performance

Model	Average Precision	ROC-AUC
Prior dummy baseline	0.1570	0.5000
Logistic Regression	**0.2348**	**0.6446**
XGBoost	0.2301	0.6368

Model selected

Logistic Regression leads validation AP at **0.2348**, compared with 0.2301 for XGBoost and 0.1570 for the prior baseline. The advantage over XGBoost is small.

Logistic Regression is selected because it has the highest validation AP. Its advantage over XGBoost is small and does not establish universal superiority. The learned models both improve on the constant-score baseline for validation AP. Without repeated temporal evaluations or uncertainty estimates, the difference should not be described as statistically significant.

7.2 Held-out test performance

The selected Logistic Regression model was evaluated on November–December using the validation-selected threshold of 0.2611.

Table 7.2: Selected model on the held-out test set

Metric	Test result
Selected model	Logistic Regression
Decision threshold	0.2611
Average Precision	0.3359
AP lift over prevalence	1.30×
ROC-AUC	0.6070
Precision	0.3415
Recall	0.4456
F1-score	0.3867
Accuracy	0.6349
Positive-class prevalence	0.2583

How to interpret the test score

Recall is **0.4456**, so fewer than half of delayed flights are detected at this threshold. Accuracy (0.6349) is below the always-negative baseline (0.7417); the threshold was selected to balance precision and recall through F1, not to maximize accuracy.

AP is approximately 1.30 times test prevalence, indicating only modest ranking improvement. Recall of 0.4456 detects fewer than half of delayed flights, while precision of 0.3415 means many flagged flights are not delayed. ROC-AUC of 0.6070 also indicates limited discrimination. Test accuracy (0.6349) is below the always-negative baseline (0.7417). This is consistent with choosing the threshold to balance precision and recall through F1 rather than maximize accuracy; an operational system should choose its threshold using the cost of missed delays and false alerts.

7.3 Feature reliance

The model-application notebook's permutation analysis identifies scheduled-hour encoding (including `hour_sin`) and carrier among the most predictive features. This aligns with the descriptive EDA, where delay rates vary by time of day and airline. Permutation importance measures model reliance, not causality; correlated inputs can share importance, and rankings may change under temporal shift.

CHAPTER 8

Discussion

8.1 Interpreting the comparison

Logistic Regression narrowly outperformed XGBoost on validation metrics. One possible interpretation is that the engineered schedule and calendar variables contain enough signal for a regularized linear model to capture part of the pattern. The recorded experiment does not establish why the difference occurs, and nonlinear models may perform differently with additional features or tuning. The result supports selecting the model according to the declared validation criterion, not a general conclusion that simpler models are always better.

8.2 Temporal variation and operational meaning

The validation delay prevalence (15.70%) is substantially below training prevalence (26.37%) and test prevalence (25.83%). This suggests that the validation months are atypically calm, which may affect model ranking and threshold selection. AP values also depend on prevalence, so split-specific base rates matter when interpreting differences. One train/validation/test division across a single year cannot establish stability across years or airports; rolling-origin validation would provide stronger evidence.

The model estimates risk rather than explaining the cause of a delay. It has no direct weather observations, does not separately predict cancellations or diversions, and depends on the validity of schedule-derived features at the previous-evening cutoff. In particular, a feature can be predictive in a retrospective dataset and still be unsuitable for deployment if it would not be known at prediction time.

CHAPTER 9

Reproducibility and Implementation

The project repository organizes work into exploratory analysis, preprocessing, and model-application notebooks [9]. The modelling workflow uses pandas and NumPy for data handling, matplotlib/seaborn for visual analysis, scikit-learn for preprocessing pipelines and metrics, XGBoost for boosted trees, and joblib for model persistence. The recorded model-application run uses random state 42. The preprocessing notebook creates chronological splits, while the model-application notebook generates the selected model, test predictions, metrics, model-comparison records, and permutation-importance results depending on the configured output directory.

For a reproducible rerun, use the same version of the source data, restart the notebook runtime, and execute the EDA, preprocessing, and model-application notebooks in order. Verify the raw row count, labelled row count, split sizes and prevalence, predictor allow-list, selected model, threshold, and test metrics. Record package versions, random seed, notebook outputs, and repository commit. Notebook outputs depend on execution state, so reported metrics should be regenerated after changes to code or data.

The repository also contains an alternative preprocessing experiment with a different model/threshold objective. Its metrics are not mixed into this report. The final reported run here is the model-application experiment that selects Logistic Regression by validation AP and threshold 0.2611 by validation F1. The supplied Colab notebooks are intended as the executable record; data access may depend on sharing permissions.

CHAPTER 10

Limitations and Future Work

The principal limitations are:

1. **No weather inputs.** Weather and other external data could improve prediction, but only if timestamped information was available by the 6:00 PM cutoff.
2. **Limited temporal and geographic scope.** Evaluate later years and external airports before generalizing the results.
3. **Missing target labels.** Cancellations and diversions are not separately modelled because the arrival-delay label is unavailable.
4. **Temporal drift.** Use rolling-origin tests and monitor prevalence and probability calibration across periods.
5. **Feature memorization and availability.** Test the model without flight number and verify that the scheduled-origin-hour count is available and leakage-free.
6. **Decision policy.** Choose a threshold using explicit operational costs or precision/recall requirements rather than F1 alone.
7. **Deep learning.** A deep-learning extension is future work; no deep-learning result is claimed in this report.

CHAPTER 11

Conclusion

This project formulates arrival-delay prediction as binary classification, using a threshold of 15 minutes and a prediction cutoff at 6:00 PM on the previous day. Of 336,776 source records, 327,346 have usable labels. The workflow uses schedule and calendar features, train-fitted preprocessing, chronological partitions, a prior baseline, Logistic Regression, and XGBoost.

Logistic Regression is selected because its validation AP (0.2348) and ROC-AUC (0.6446) are slightly higher than XGBoost's. On the November–December holdout, the selected model achieves AP 0.3359, ROC-AUC 0.6070, precision 0.3415, recall 0.4456, and F1 0.3867 at threshold 0.2611. These results indicate limited predictive signal, not high-reliability forecasting. Test accuracy is below that of an always-negative rule, illustrating why the decision metric and operational costs must be stated explicitly. Avoiding post-departure leakage, reporting split-specific base rates, and maintaining an untouched test period are central to the validity of the experiment.

Final takeaway

The project establishes a reproducible baseline workflow and identifies modest predictive signal. Before practical use, the next priorities are stronger temporal validation, confirmation that every feature is available at the cutoff, weather/context data where permitted, and a threshold tied to real operating costs.

CHAPTER A

Reproducibility Checklist

1. Confirm that the raw dataset has 336,776 rows and 21 columns, including 9,430 missing arrival-delay labels.
2. Confirm that 327,346 labelled records remain after target cleaning.
3. Recreate the chronological split sizes and delay rates shown in Table 6.1.
4. Exclude post-departure outcomes and target-related columns from predictors.
5. Fit imputation, scaling, and encoding on training data only.
6. Verify the selected model and threshold using validation data only.
7. Apply threshold 0.2611 unchanged to the held-out test predictions.
8. Regenerate and verify all reported metrics after any code/data change.
9. Record Python and package versions, random seed, outputs, and repository commit.

References

[1] M. Sajadi, "Flights," Kaggle dataset, 2013 New York City flight records. Available: https://www.kaggle.com/datasets/matinsajadi/flights. Accessed Oct. 22, 2026.

[2] H. Wickham, "nycflights13: Data about flights departing NYC in 2013," R package, 2014. Available: https://github.com/tidyverse/nycflights13.

[3] S. Gharibzadeh, M. M. Pedram, M. Khademi, and A. Salajegheh, "Trends and gaps in machine learning based flight delay prediction in spatiotemporal and data-driven approaches: A systematic review," *Applied Soft Computing*, vol. 202, Art. no. 115761, 2026, doi: 10.1016/j.asoc.2026.115761.

[4] H. da Silva Queiróz Júnior et al., "Machine learning methods benchmarking for predicting flight delays: An efficiency meta-analysis," *Sustainability*, vol. 17, no. 21, Art. no. 9887, 2025, doi: 10.3390/su17219887.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[6] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006, doi: 10.1016/j.patrec.2005.10.010.

[7] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[8] Scikit-learn Developers, "Common pitfalls and recommended practices: Data leakage." Available: https://scikit-learn.org/stable/common_pitfalls.html. Accessed Oct. 22, 2026.

[9] abhi-7-7, "Flight Delay Prediction – NYC Flights (2013)," GitHub repository. Available: https://github.com/abhi-7-7/Flight_Delay_Prediction. Accessed Oct. 22, 2026.